Broken Access Control: Admin Section OWASP Juice Shop

Security Severity: High (Potential)

1. Concept Overview

Broken Access Control occurs when an application fails to properly enforce authorization restrictions, allowing users to access functionality beyond their permitted privileges.
In this challenge, the objective was to access the hidden Administration section of OWASP Juice Shop.

2. Vulnerability Identification

The Administration section was not directly available through the normal user interface.
When attempting to access the route directly as an unauthenticated or regular user, the application displayed an Unauthorized alert.

3. Methodology — Step-by-Step

Step 1: Discover the Administration Route

- Opened OWASP Juice Shop.
- Entered the following route directly into the browser: http://localhost:3000/#/administration

Observed an Unauthorized alert, indicating that access was restricted.

Step 2: Bypass Login Authentication

- Navigated to the login page.
- Identified the SQL Injection vulnerability in the authentication functionality.
- Used SQL Injection to bypass the login mechanism.
- Successfully authenticated as the administrator without knowing the actual administrator password.

Step 3: Access the Administration Section

- After obtaining administrator authentication, entered the administration route again: http://localhost:3000/#/administration
- Successfully accessed the Administration section.
- Confirmed that the administrative interface was accessible.

Step 4: Verify Challenge Completion

- Opened the OWASP Juice Shop Score Board.
- Verified that the Admin Section challenge was marked as completed.

4. Technical Explanation

Two security concepts were demonstrated:

- **SQL Injection:** Manipulating authentication queries to bypass login and obtain administrator access.
- **Broken Access Control:** Accessing privileged administrative functionality that should be restricted to authorized users.

The SQL Injection provided administrator authentication; the Administration route exposed the privileged interface after authentication.

5. Security Impact

If these weaknesses exist in a real application, an attacker may potentially:

- Bypass authentication.
- Gain administrator-level access.
- View sensitive administrative information.
- Modify or delete application data.
- Perform unauthorized administrative operations.

6. Remediation

- Use parameterized SQL queries to prevent SQL Injection.
- Implement secure authentication mechanisms.
- Enforce server-side role-based access control.
- Validate administrator permissions for every privileged API request.
- Apply least-privilege principles.
- Monitor suspicious authentication and authorization attempts.

7. Result

Successfully completed the Admin Section challenge by exploiting SQL Injection to authenticate as the administrator and accessing the hidden Administration section through its direct URL.

8. Evidence Screenshots

Include screenshots showing:

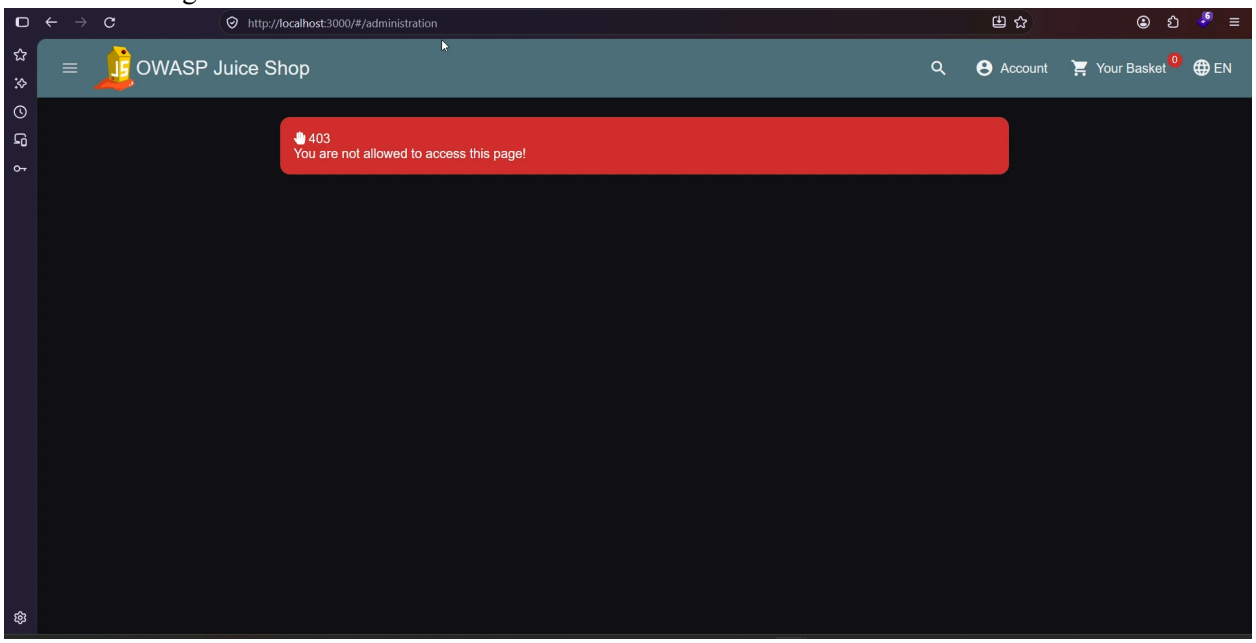


Fig: Unauthorized alert when accessing the Administration route.

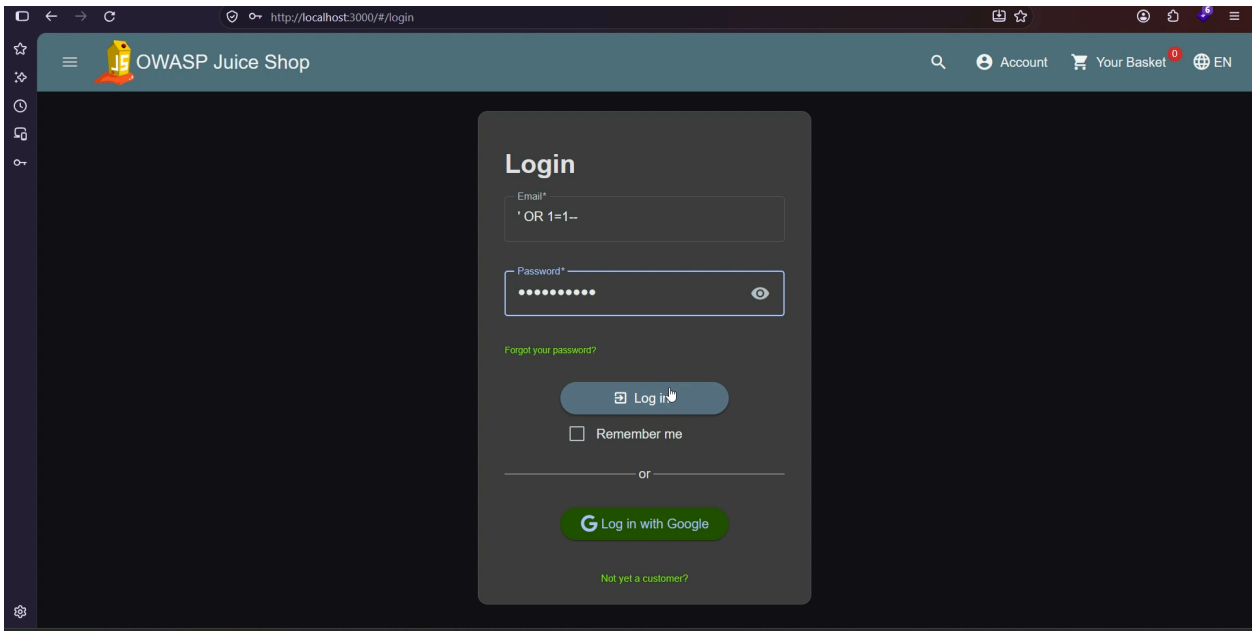


Fig: SQL Injection authentication process.

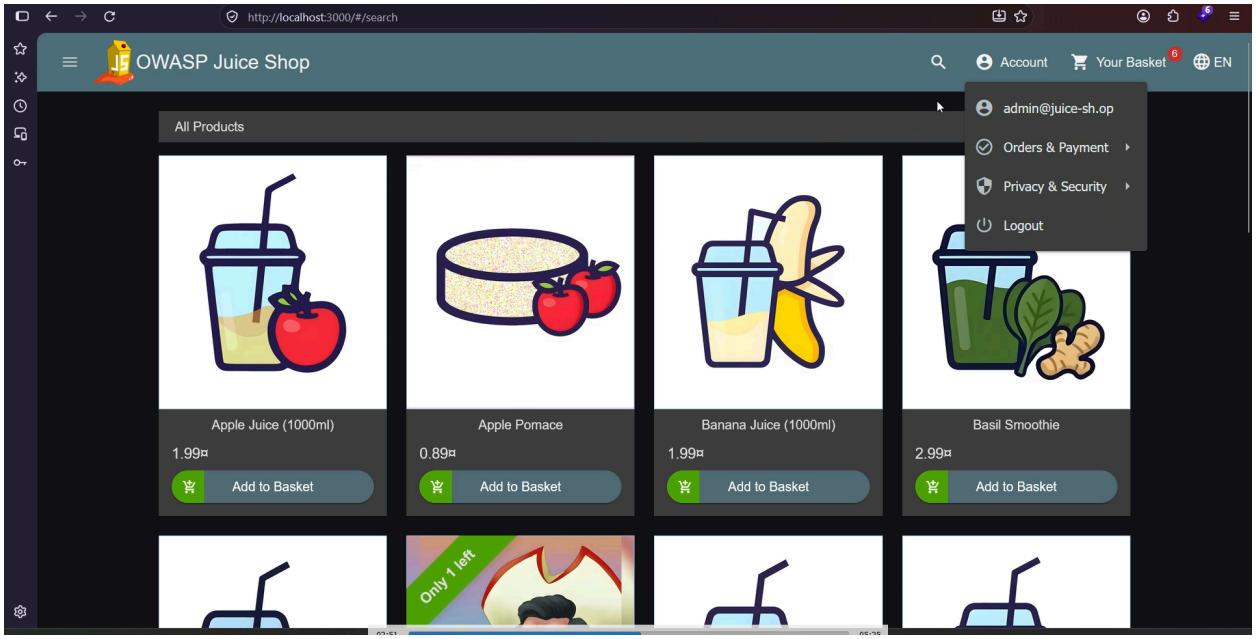


Fig: Successful administrator login.

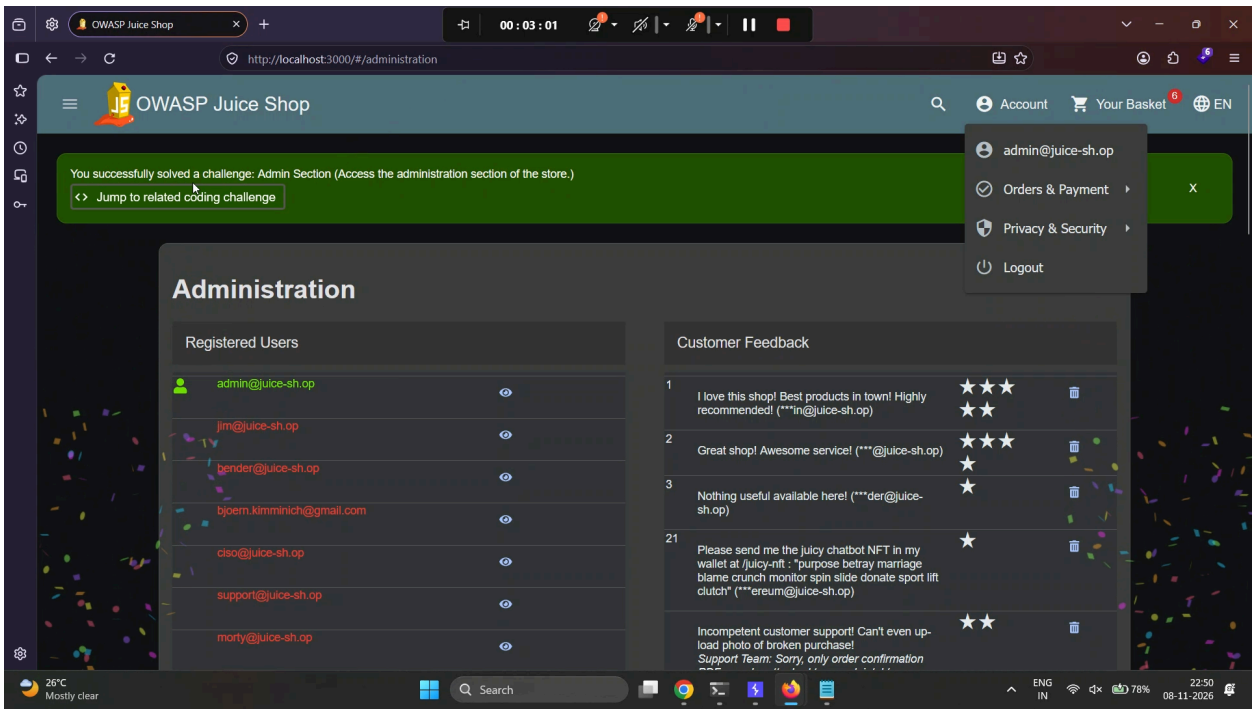


Fig: Administration dashboard.

9. References

- OWASP Juice Shop — Broken Access Control
- OWASP SQL Injection Prevention Cheat Sheet
- OWASP Authorization Cheat Sheet

Severity note: High is a suggested potential severity because the demonstrated SQL Injection enabled administrator authentication. The official Juice Shop difficulty is 2 stars, not a CVSS rating.

Coding Challenge: Admin Section

Find It & Fix It completed

1. Find It — Identify the Vulnerability

Inspected the application's route configuration and identified the administration route:

```
{
  path: 'administration',
  component: AdministrationComponent,
  canActivate: [AdminGuard]
}
```

Explanation:

- path: 'administration' defines the administration route.
- AdministrationComponent loads the admin interface.
- canActivate: [AdminGuard] applies an authorization guard before route access.

The challenge highlights the exposure of administrative functionality within the public-facing application.

2. Fix It — Apply the Correct Solution

Selected Fix: 3
Correct solution
Fix 3 comments out the administration route and introduces the following architectural recommendation:

```
/*
TODO: Externalize admin functions into separate application
that is only accessible inside corporate network.
*/
```

Explanation:
The suggested solution follows the Separation of Concerns principle.

- Separate administrative functionality from the public-facing application.
- Host the admin backend as a distinct application.
- Restrict access to the internal corporate network.
- Reduce exposure of privileged functionality to public users.

3. Why Fix 3 Is Correct

A client-side route guard is useful, but it should not be the only security control protecting administrative functionality.
Moving administrative functions into a separate, restricted application reduces their exposure. Server-side authorization must still protect all administrative APIs and operations.

4. Result

Successfully completed both stages:

- **Find It:** Identified the administration route and its access-control configuration.
- **Fix It:** Selected Fix 3 to separate administrative functionality from the public-facing application.
- Received the green success confirmation.

5. Evidence Screenshots

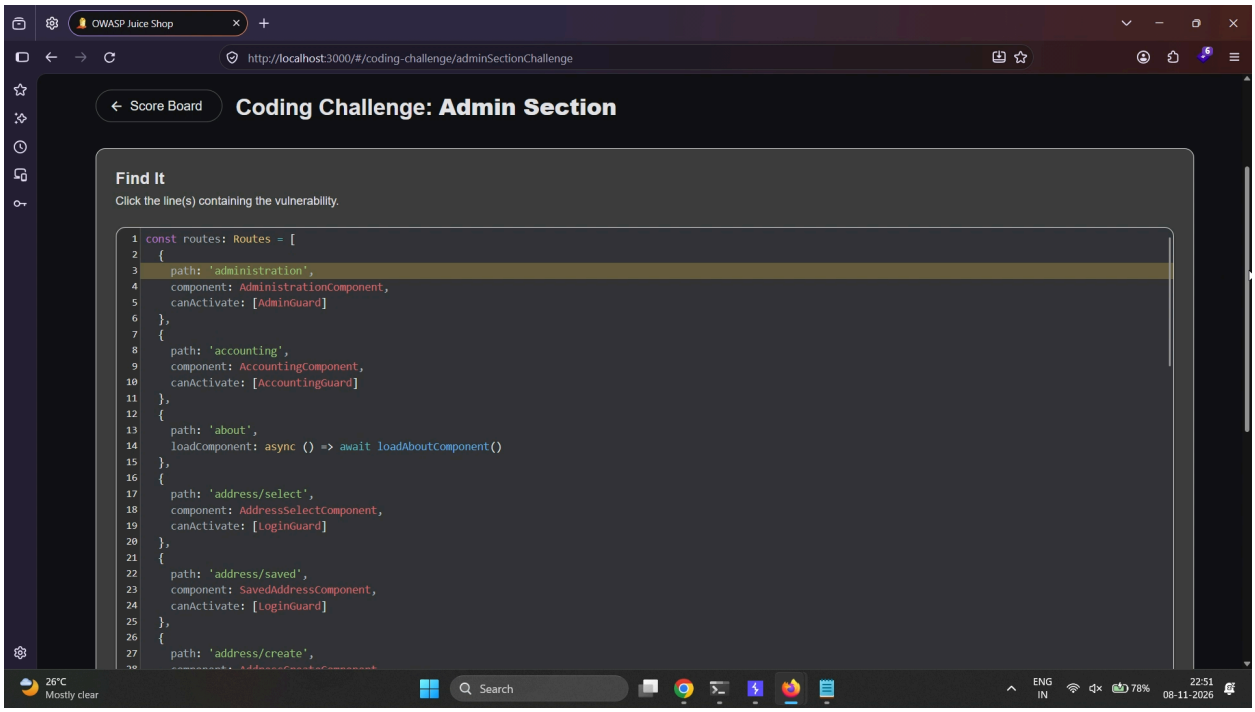


Fig 1: Administration route configuration showing AdminGuard.

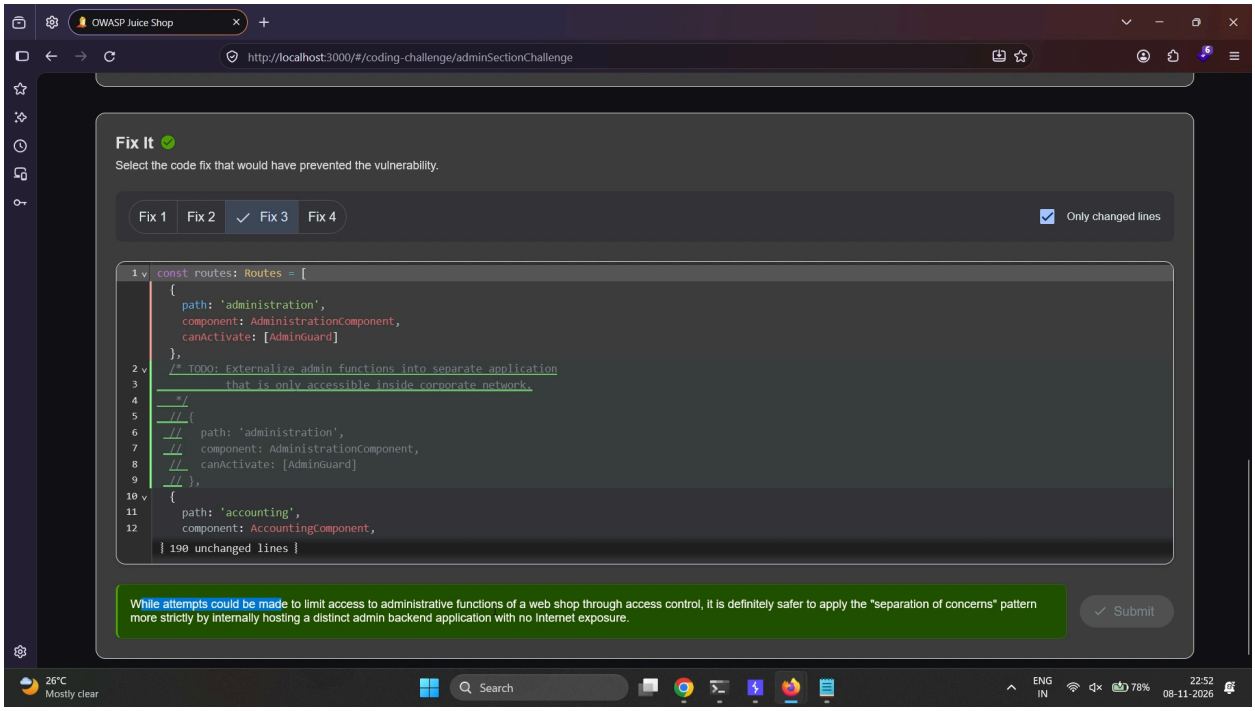


Fig 2: Fix 3 selected with the green success confirmation.

Key Learning: Do not rely exclusively on client-side route guards to protect sensitive administrative functionality. Use server-side authorization and secure architectural separation.